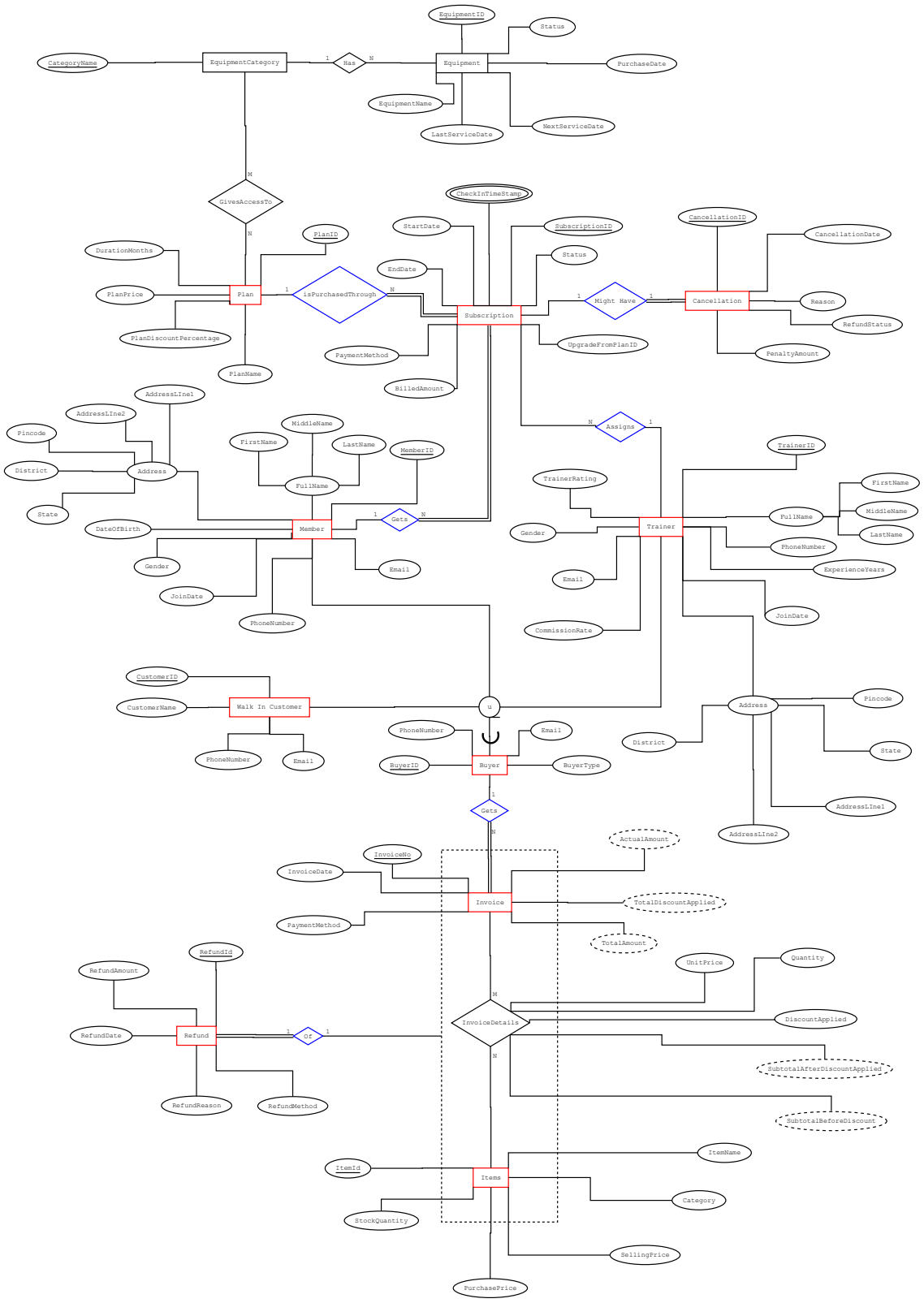
SCHEMA DIAGRAM

FitZone Gym Management System

The following page shows the complete ERD
depicting all entities, attributes, and relationships.

Entity Relationship Diagram

FitZone Gym Management System — IT-214, Lab Group G2, Section A



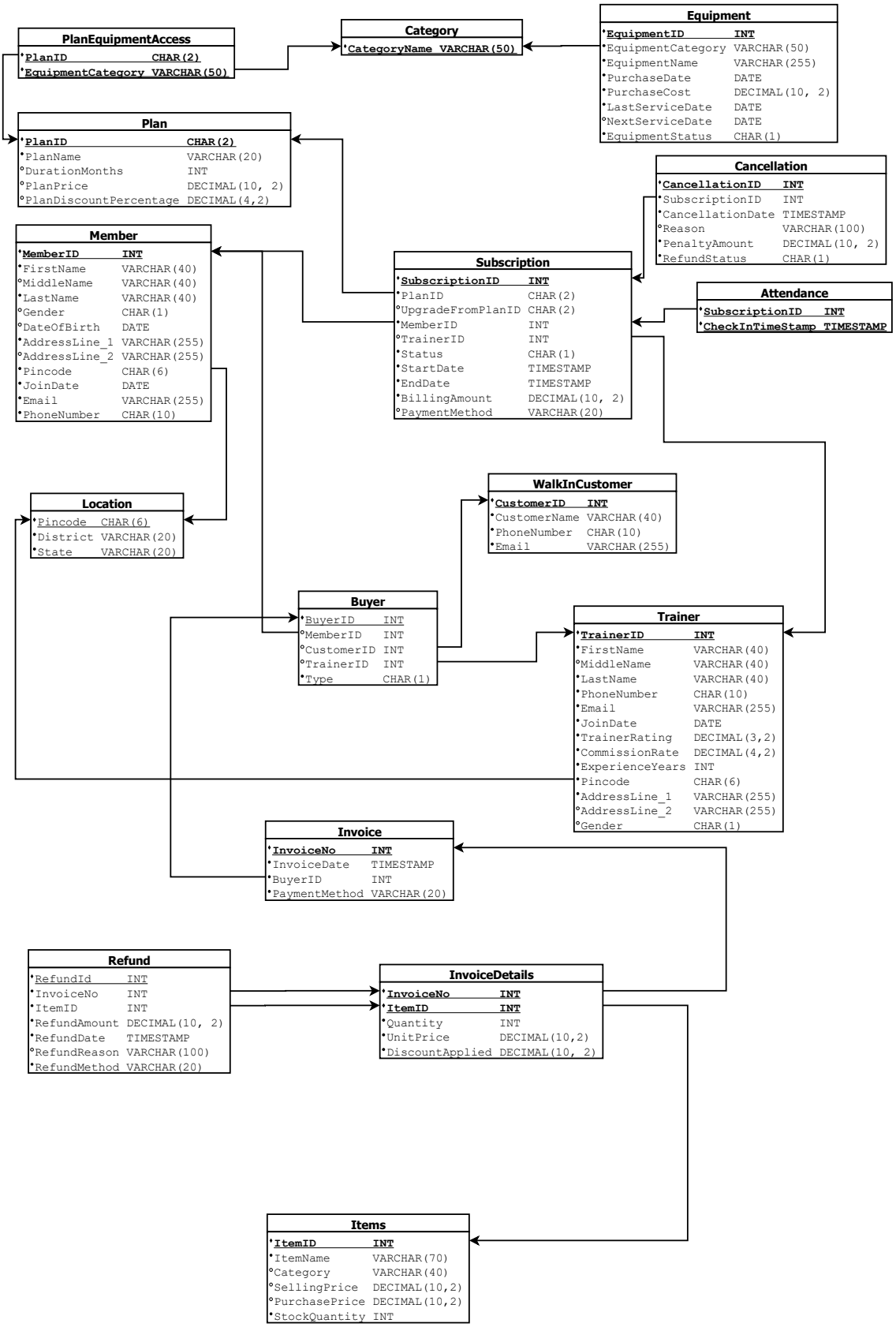
SCHEMA DIAGRAM

FitZone Gym Management System

The following page shows the complete Relational Schema
with all tables, attributes, and data types.

Relational Schema

FitZone Gym Management System — IT-214, Lab Group G2, Section A



Contents

1 Introduction

This report presents a formal verification that the relational schema of the **FitZone Gym Management System** satisfies **Boyce–Codd Normal Form (BCNF)** — the strongest practical normal form in classical relational database theory. Every table in the schema is analysed by identifying all non-trivial functional dependencies and confirming that the determinant in each case is a superkey.

What is BCNF?

A relation schema R is in **Boyce–Codd Normal Form** if and only if, for every non-trivial functional dependency of the form:

$$X \to Y \quad \text{where } Y \not\subseteq X$$

X is a **superkey** of R. That is, X functionally determines every attribute in R. BCNF implies that every fact in the relation is a fact about the key, the whole key, and nothing but the key — eliminating all update, insertion, and deletion anomalies.

Notation Used in This Report

`Attribute`	Superkey / Primary Key attribute
Attribute	Primary Key attribute
Attribute	Foreign Key attribute
$X \to Y$	X functionally determines Y
✓ **BCNF**	Table satisfies BCNF

2 Schema Overview

The FitZone schema consists of **16 relations** grouped into four functional domains:

#	Table	Domain	Purpose
1	**Location**	Infrastructure	Stores pincode, district, state
2	**Member**	People	Registered gym members
3	**Trainer**	People	Gym trainers and coaches
4	**WalkInCustomer**	People	Walk-in retail customers
5	**Plan**	Membership	Subscription plan definitions
6	**Category**	Inventory	Equipment category names
7	**Equipment**	Inventory	Physical gym equipment
8	**PlanEquipmentAcc**	Membership	Plan-to-category access mapping
9	**Items**	Inventory	Retail products and supplements
10	**Buyer**	Commerce	Polymorphic buyer abstraction
11	**Subscription**	Membership	Member subscription records
12	**Cancellation**	Membership	Subscription cancellation records
13	**Attendance**	Operations	Gym check-in log
14	**Invoice**	Commerce	Retail sale invoices
15	**InvoiceDetails**	Commerce	Line items per invoice
16	**Refund**	Commerce	Item refund records

3 BCNF Verification — Table by Table

3.1 Location

Schema: Location(Pincode, District, State)

Functional Dependencies

`Pincode` → `District, State` `Pincode` is the Primary Key

Verdict: The only non-trivial FD has `Pincode` as its determinant, which is the primary key and therefore a superkey. ✓ **BCNF**

Design Note: This table was deliberately extracted from **Member** and **Trainer** to eliminate the transitive dependency `Pincode` → `District` → `State` that would have existed if address fields were stored inline. This is a key normalisation decision.

3.2 Member

Schema: Member(MemberID, FirstName, MiddleName, LastName, Gender, DateOfBirth, AddressLine_1, AddressLine_2, *Pincode*, JoinDate, Email, PhoneNumber)

Functional Dependencies

`MemberID` → All attributes `MemberID` is the PK

`Email` → All attributes `Email` is a Candidate Key

`PhoneNumber` → All attributes `PhoneNumber` is a Candidate Key

Verdict: Every determinant (`MemberID`, `Email`, `PhoneNumber`) is a candidate key, hence a superkey. No non-key attribute determines any other non-key attribute. ✓ **BCNF**

3.3 Trainer

Schema: Trainer(TrainerID, FirstName, MiddleName, LastName, PhoneNumber, Email, Gender, *Pincode*, AddressLine_1, AddressLine_2, JoinDate, TrainerRating, CommissionRate, ExperienceYears)

Functional Dependencies

`TrainerID` → All attributes — `TrainerID` is the PK

`Email` → All attributes — `Email` is a Candidate Key

`PhoneNumber` → All attributes — `PhoneNumber` is a Candidate Key

Verdict: All three determinants are candidate keys and therefore superkeys. ✓ **BCNF**

3.4 WalkInCustomer

Schema: WalkInCustomer(CustomerID, CustomerName, PhoneNumber, Email)

Functional Dependencies

`CustomerID` → All attributes — `CustomerID` is the PK

`Email` → All attributes — `Email` is a Candidate Key (UNIQUE)

`PhoneNumber` → All attributes — `PhoneNumber` is a Candidate Key (UNIQUE)

Verdict: All determinants are candidate keys. ✓ **BCNF**

3.5 Plan

Schema: Plan(PlanID, PlanName, DurationMonths, PlanPrice, PlanDiscountPercentage)

Functional Dependencies

`PlanID` → All attributes — `PlanID` is the PK

`PlanName` → All attributes — `PlanName` is a Candidate Key (UNIQUE)

Verdict: Both determinants are candidate keys. ✓ **BCNF**

3.6 Category

Schema: Category(CategoryName)

Functional Dependencies

Only the trivial dependency exists: `CategoryName` → `CategoryName`.

Verdict: A single-attribute relation with no non-trivial FDs. Trivially in BCNF. ✓ **BCNF**

3.7 Equipment

Schema: Equipment(EquipmentID, EquipmentName, *EquipmentCategory*, PurchaseCost, PurchaseDate, LastServiceDate, NextServiceDate, EquipmentStatus)

Functional Dependencies

`EquipmentID` → All attributes `EquipmentID` is the PK

Potential concern: `EquipmentCategory` is a foreign key to **Category**, but within **Equipment** it does not determine any other attribute. It merely classifies the equipment. No transitive dependency arises. ✓ **BCNF**

3.8 PlanEquipmentAccess

Schema: PlanEquipmentAccess(*PlanID*, *EquipmentCategory*)

Functional Dependencies

This is a pure junction (many-to-many) table with **no non-key attributes**. The only functional dependencies present are trivial.

Verdict: A relation with no non-key attributes is trivially in BCNF. ✓ **BCNF**

3.9 Items

Schema: Items(ItemID, ItemName, Category, SellingPrice, PurchasePrice, StockQuantity)

Functional Dependencies

`ItemID` → All attributes `ItemID` is the Primary Key

Potential concern: `Category` is a plain `VARCHAR(40)` label stored within this table. One might ask whether it creates a transitive dependency (e.g., `Category` → some other attribute). It does **not** — no non-key attribute in **Items** depends on `Category` alone. `Category` merely classifies the item; it does not determine `SellingPrice`, `PurchasePrice`, or `StockQuantity`.

Note: `Category` here is a plain descriptive string, distinct from the **Category** table used for equipment. It carries no FK constraint and introduces no functional dependency beyond `ItemID` → `Category`. ✓ **BCNF**

3.10 Buyer

Schema: Buyer(BuyerID, *MemberID*, *CustomerID*, *TrainerID*, Type)

Functional Dependencies

`BuyerID` → All attributes — `BuyerID` is the PK

`MemberID` → All attributes — `MemberID` is a Candidate Key (UNIQUE)

`CustomerID` → All attributes — `CustomerID` is a Candidate Key (UNIQUE)

`TrainerID` → All attributes — `TrainerID` is a Candidate Key (UNIQUE)

Design Note: `Type` is technically co-determined by which of the three FK columns is non-null. This is enforced at the constraint level (`chk_buyer_exclusive`), not as a stored FD. All determinants remain superkeys. ✓ **BCNF**

3.11 Subscription

Schema: Subscription(SubscriptionID, *PlanID*, UpgradeFromPlanID, *MemberID*, *TrainerID*, Status, BillingAmount, PaymentMethod, StartDate, EndDate)

Functional Dependencies

`SubscriptionID` → All attributes — `SubscriptionID` is the PK

Key design decisions:

- `BillingAmount` stores the *actual amount billed at time of purchase*, not derived from `PlanPrice`. This is intentional — prices may change over time — so no transitive dependency exists.
- `UpgradeFromPlanID` is a nullable FK to **Plan**. It records history, not a current dependency.
- The composite UNIQUE constraint `(MemberID, PlanID, StartDate)` forms an additional candidate key.

✓ **BCNF**

3.12 Cancellation

Schema: Cancellation(CancellationID, *SubscriptionID*, CancellationDate, Reason, PenaltyAmount, RefundStatus)

Functional Dependencies

`CancellationID` → All attributes `CancellationID` is the PK

`SubscriptionID` → All attributes `SubscriptionID` is a Candidate Key (UNIQUE)

Verdict: Both determinants are candidate keys. One cancellation can exist per subscription (1:1 enforced). ✓ **BCNF**

3.13 Attendance

Schema: Attendance(*SubscriptionID*, CheckInTimeStamp)

Functional Dependencies

This relation has **no non-key attributes**. The composite primary key `(SubscriptionID, CheckInTimeStamp)` is the entire relation. Only trivial FDs exist.

Verdict: Trivially in BCNF. ✓ **BCNF**

3.14 Invoice

Schema: Invoice(InvoiceNo, InvoiceDate, PaymentMethod, *BuyerID*)

Functional Dependencies

`InvoiceNo` → All attributes `InvoiceNo` is the PK

Verdict: Single candidate key, no non-key dependencies. ✓ **BCNF**

3.15 InvoiceDetails

Schema: InvoiceDetails(*InvoiceNo*, *ItemID*, Quantity, UnitPrice, DiscountApplied)

Functional Dependencies

`(InvoiceNo, ItemID)` → All attributes Composite PK is the superkey

Potential concern: One might ask whether `UnitPrice` should be derived from `Items.SellingPrice`. It is **intentionally stored here** to capture the price at the time of sale, independent of future price changes. Therefore `ItemID` alone does **not** determine `UnitPrice` in this table, and no BCNF violation exists. ✓ **BCNF**

3.16 Refund

Schema: Refund(RefundId, *InvoiceNo*, *ItemID*, RefundMethod, RefundAmount, RefundDate, RefundReason)

Functional Dependencies

`RefundId` → All attributes — `RefundId` is the PK

`(InvoiceNo, ItemID)` → All attributes — Candidate Key (UNIQUE constraint)

Verdict: Both determinants are candidate keys. One refund is allowed per line item per invoice. ✓ **BCNF**

4 Summary of BCNF Verification

#	Table	Superkeys / Candidate Keys	BCNF
1	**Location**	Pincode	✓
2	**Member**	MemberID, Email, PhoneNumber	✓
3	**Trainer**	TrainerID, Email, PhoneNumber	✓
4	**WalkInCustomer**	CustomerID, Email, PhoneNumber	✓
5	**Plan**	PlanID, PlanName	✓
6	**Category**	CategoryName	✓
7	**Equipment**	EquipmentID	✓
8	**PlanEquipmentAcce**	(PlanID, EquipmentCategory)	✓
9	**Items**	ItemID	✓
10	**Buyer**	BuyerID, MemberID, CustomerID, TrainerID	✓
11	**Subscription**	SubscriptionID, (MemberID, PlanID, StartDate)	✓
12	**Cancellation**	CancellationID, SubscriptionID	✓
13	**Attendance**	(SubscriptionID, CheckInTimeStamp)	✓
14	**Invoice**	InvoiceNo	✓
15	**InvoiceDetails**	(InvoiceNo, ItemID)	✓
16	**Refund**	RefundId, (InvoiceNo, ItemID)	✓

✓ **All 16 tables satisfy Boyce–Codd Normal Form**

The FitZone Gym Management System schema is fully normalised to BCNF.

5 Key Design Decisions Enabling BCNF

The following design choices were the critical structural decisions that ensure the schema remains in BCNF:

1. Location Table — Eliminating Transitive Dependency

Without the **Location** table, both **Member** and **Trainer** would have contained:

$$\text{MemberID} \rightarrow \text{Pincode} \rightarrow \text{District, State}$$

This is a transitive dependency (non-superkey `Pincode` determines `District` and `State`), which violates BCNF. Extracting **Location**(Pincode, District, State) removes this violation entirely.

2. PlanEquipmentAccess — Resolving M:N Relationship

A many-to-many relationship between **Plan** and **Category** (a plan grants access to multiple equipment categories; a category can be accessed by multiple plans) is correctly resolved into the junction table **PlanEquipmentAccess**(PlanID, EquipmentCategory). Storing this inline would have caused repeating groups, violating even 1NF.

3. Buyer Table — Polymorphic Abstraction

The **Buyer** table acts as a polymorphic supertype, allowing **Invoice** to reference a single `BuyerID` regardless of whether the buyer is a **Member**, **Trainer**, or **WalkInCustomer**. The exclusive CHECK constraint ensures exactly one FK column is populated. Without this table, **Invoice** would require three nullable FK columns, introducing potential partial dependencies.

4. Point-in-Time Pricing — Preventing Spurious Dependencies

`BillingAmount` in **Subscription** and `UnitPrice` in **InvoiceDetails** store values *at the time of transaction*. If these were derived from **Plan.PlanPrice** and **Items.SellingPrice** respectively, updating a price would cause historical records to silently change — a classic update anomaly. Storing them independently also means `PlanID` alone does not determine `BillingAmount`, preventing a BCNF violation.

5. Category Table — Avoiding Repeating Strings

Storing `EquipmentCategory` as a VARCHAR in **Equipment** without a lookup table would have made it impossible to enforce referential integrity. The **Category** table ensures every category name is valid and consistent, while the FK in **Equipment** and **PlanEquipmentAccess** prevents orphaned values.

6 Conclusion

Final Statement

The **FitZone Gym Management System** relational schema, comprising **16 tables**, has been formally verified to satisfy **Boyce–Codd Normal Form** across all relations.

For every non-trivial functional dependency $X \to Y$ identified in every table, X is a superkey of that relation. The schema contains:

- **No partial dependencies** — all non-key attributes depend on the full primary key
- **No transitive dependencies** — no non-key attribute determines another non-key attribute
- **No BCNF violations** — every determinant in every non-trivial FD is a superkey

This guarantees the schema is free from **update**, **insertion**, and **deletion anomalies**, providing a robust and consistent foundation for the FitZone Gym Management System.